FIRST READ APP THEN TRAIN

Your Flask file does:

```
Receive email
↓
Use trained model
↓
Predict SPAM/HAM
```

This file does:

```
Read dataset
↓
Train model
↓
Test accuracy
↓
Save model
```

Think:

```
train_model.py
```

creates the brain.

```
app.py
```

uses the brain.

◆ IMPORT PANDAS

```
import pandas as pd
```

import → keyword

pandas → external library

as → keyword

pd → alias (short name)

👉 Use: work with datasets and tables.

◆ IMPORT PICKLE

```
import pickle
```

pickle → built-in module

👉 Use: save and load Python objects.

Later used to save:

```
spam_model.pkl
```

and

```
vectorizer.pkl
```

◆ IMPORT RE

```
import re
```

re → Regular Expression module

👉 Use: find, match, replace, and clean text using patterns.

◆ CLEAN TEXT FUNCTION

```
def clean_text(text):
```

def → keyword

clean_text → function name

text → parameter

👉 Use: define a function that cleans text before training.

◆ CONVERT TO LOWERCASE

```
text = text.lower()
```

text → variable

lower() → String method

👉 Use: convert all letters to lowercase.

Example:

```
"FREE MONEY"
```

becomes:

```
"free money"
```

◆ REMOVE SPECIAL CHARACTERS

```
text = re.sub(r'[^a-z0-9 ]', '', text)
```

re → Regular Expression module

sub() → substitute (replace) method

r'[^a-z0-9]' → pattern

'' → replace with nothing

text → source text

👉 Use: remove special characters.

Example:

```
"Win $1000!!!"
```

becomes:

```
"win 1000"
```

◆ RETURN CLEANED TEXT

```
return text
```

return → keyword

text → cleaned text

👉 Use: send cleaned text back to whoever called the function.

◆ LOAD DATASET

```
df = pd.read_csv("spam.csv")
```

df → variable

pd → pandas alias

read_csv() → pandas function

"spam.csv" → dataset file

👉 Use: load CSV data into a DataFrame.

Think:

```
spam.csv
↓
DataFrame
↓
df
```

◆ CONVERT LABELS

```
df['Category'] = df['Category'].map({'ham': 0, 'spam': 1})
```

df → DataFrame

['Category'] → column name

map() → method

{'ham':0,'spam':1} → mapping dictionary

👉 Use: convert text labels into numbers.

Before:

```
ham
spam
ham
```

After:

```
0
1
0
```

◆ CLEAN MESSAGES COLUMN

```
df['Messages'] = df['Messages'].apply(clean_text)
```

apply() → method

clean_text → function

👉 Use: run clean_text() on every message.

Example:

```
"WIN MONEY!!!"
```

becomes:

```
"win money"
```

for every row in the dataset.

◆ FEATURES

```
X = df['Messages']
```

X → variable

Messages → input column

👉 Use: store email/message text.

◆ TARGET

```
y = df['Category']
```

y → variable

Category → output column

👉 Use: store correct answers.

Example:

```
Message           Category

Hi friend            0
Win money now        1
```

X:

```
Messages
```

y:

```
Category
```

◆ CREATE TF-IDF VECTORIZER

```
vectorizer = TfidfVectorizer(stop_words='english') 📕/ check vectorizer pdf
```

vectorizer → variable

TfidfVectorizer → sklearn class

stop_words → parameter

english → built-in English stop-word list

👉 Use: convert text into numerical features.

◆ WHAT ARE STOP WORDS?

Common words:

```
the
is
am
are
and
of
to
```

Usually don't help detect spam.

So TF-IDF removes many of them.

◆ LEARN VOCABULARY + CONVERT TEXT

```
X_vectorized = vectorizer.fit_transform(X) // check fit & transform
```

fit() → learn words

transform() → convert text to numbers

fit_transform() → both together

X → message column

👉 Use: learn vocabulary and convert messages into numerical vectors.

◆ SPLIT DATA

```
X_train, X_test, y_train, y_test =  // check test
train_test_split(
    X_vectorized,
    y,
    test_size=0.2,
    random_state=42
)
```

IMP.

train_test_split() → sklearn function

test_size=0.2 → 20% testing

random_state=42 → fixed random split // check random 42

👉 Use: divide data into training and testing sets.

◆ WHAT IS 80-20 SPLIT?

Suppose:

```
1000 messages
```

Training:

```
800
```

Testing:

```
200
```

Model learns from 800.

Model is evaluated on 200.

◆ CREATE MODEL

```
model = MultinomialNB(alpha=0.1) // check multinomialnb
```

model → variable

MultinomialNB → Naive Bayes classifier

alpha=0.1 → smoothing parameter

👉 Use: create spam-classification model.

◆ TRAIN MODEL

```
model.fit(X_train, y_train)
```

fit() → training method

X_train → input data

y_train → correct answers

👉 Use: teach the model using training data.

Think:

```
Messages
+
Correct Labels
↓
Model Learns Patterns
```

◆ MAKE PREDICTIONS

```
y_pred = model.predict(X_test)
```

y_pred → variable

predict() → method

X_test → unseen test data

👉 Use: predict labels for test messages.

◆ CALCULATE ACCURACY

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

accuracy_score() → sklearn function

y_test → actual answers

y_pred → predicted answers

👉 Use: calculate prediction accuracy.

Example:

```
Actual:
[1,0,1,1]

Predicted:
[1,0,1,0]
```

Correct:

```
3 out of 4
```

Accuracy:

```
75%
```

◆ SAVE MODEL

```
pickle.dump(model, open("spam_model.pkl", "wb"))  📖
```

dump() → save object

model → trained model

"spam_model.pkl" → file name

wb → write binary

👉 Use: save trained model to disk.

◆ SAVE VECTORIZER

```
pickle.dump(vectorizer, open("vectorizer.pkl", "wb"))  📄
```

vectorizer → TF-IDF object

👉 Use: save learned vocabulary and TF-IDF configuration.

◆ SUCCESS MESSAGE

```
print("Model saved successfully!")
```

print() → function

👉 Use: show confirmation that files were saved.

🔥 COMPLETE FLOW

```
spam.csv
↓
Load Dataset
↓
Clean Text
↓
Convert ham/spam → 0/1
↓
TF-IDF Vectorization
↓
Train/Test Split
↓
Train Naive Bayes Model
↓
Calculate Accuracy
↓
Save Model (.pkl)
↓
Flask Loads Model Later
↓
Spring Boot Sends Email
```

```
↓
Prediction Returned
```